

Cost-Aware Eviction for LLM Response Caching: A GDSF-Based Approach for GPTCache

Author: Nissim Brami (nissimbrami@post.bgu.ac.il) **Course:** Caching in LLMs — Ben-Gurion University of the Negev **Repository:** <https://github.com/nissimbrami/cost-aware-eviction-gptcache>

Abstract

LLM API calls are among the most expensive operations in modern AI stacks: per-query costs of a few cents accumulate quickly at scale. Semantic caching systems such as GPTCache reduce those costs by reusing prior responses, but their default eviction policies (LRU and FIFO) treat every cached entry as equally valuable — which they are not. A cached response for an expensive GPT-4 completion is worth far more than a cached response for a short GPT-3.5 answer. In this project I implement a cost-aware eviction policy based on the Greedy-Dual-Size-Frequency (GDSF) algorithm from the classical web-proxy caching literature, and evaluate it as a plugin for GPTCache. Each entry receives priority $\text{Priority}(i) = \text{Clock} + \text{freq}(i)^\alpha \cdot \text{cost}(i)^\beta / \text{size}(i)$, where the clock advances to the priority of the last evicted item, providing implicit aging. Across six synthetic workloads and four cache sizes (30 runs each, 3600 experiments), GDSF delivers **+25.7% dollar savings on high-variance-cost traffic**, **+32.2% on bursty traffic**, **+91.1% on size-varying traffic**, and **+18.5% on the anti-LRU adversarial pattern**, while matching LRU on uniform-cost and Zipf-fits-in-cache workloads. The code, benchmark harness, ablation study, and 259-test suite are open-source under the MIT license.

1. Introduction

I chose this project because the "value" of a cache entry in an LLM system is directly observable in the invoice from OpenAI, Anthropic, or whoever the inference provider is. Unlike traditional caches — where a hit saves a few milliseconds of CPU — an LLM cache hit saves real money, and the amount saved varies by an order of magnitude between short cheap prompts and long expensive completions. That makes cost-aware eviction not a theoretical curiosity but the most economically important lever available to a cache designer in this domain.

GPTCache [1] currently ships only LRU and FIFO. Both are recency-only policies that ignore per-entry cost, frequency, and size. Consider a cache at capacity that must evict one of two entries: (A) a short response that cost 0.01 to regenerate and is rarely accessed, or (B) a long completion that cost 0.10 to regenerate and is frequently requested. LRU picks the older one; LFU picks the less-accessed one; neither is aware that evicting B is ten times more expensive to recover than evicting A. GDSF folds all three dimensions — frequency, cost, size — into a single priority score with a clean theoretical grounding [4, 8].

Contributions

1. A **GDSF-based eviction policy** adapted for LLM response caching, using token-count $\times$ per-model pricing to estimate per-entry cost.
1. An **implementation as a drop-in EvictionBase plugin for GPTCache**, with $O(\log n)$ per-operation complexity via an indexed min-heap and full thread safety.
1. A **comprehensive benchmark harness** — 6 workloads $\times$ 5 policies $\times$ 4 cache sizes $\times$ 30 runs (~3600 experiments) — with statistical rigor (paired t-tests, Bonferroni correction, BCa bootstrap 95% CIs).
1. An $\alpha \times \beta$ **ablation study** confirming the policy is robust to parameter choice within a wide operating range.
1. A **259-test suite** including 6 Hypothesis-based property tests that

surfaced a real edge case in capacity handling under duplicate keys.

The remainder of this report: Section 2 covers related work, Section 3 the system design, Section 4 the experimental setup, Section 5 the results, Section 6 the discussion, Section 7 conclusions and future work.

2. Background and Related Work

2.1 LLM Caching Systems

The high cost and latency of LLM inference has motivated several caching approaches at different levels of the serving stack.

GPTCache [1] is an open-source semantic caching library that intercepts LLM API calls, embeds the query using a lightweight encoder (e.g., ONNX-based sentence transformers), searches a vector store for semantically similar prior queries, and returns cached responses when similarity exceeds a configurable threshold. GPTCache provides a modular architecture with pluggable components for embedding, similarity evaluation, storage, and eviction. Its default eviction policies are LRU and FIFO, implemented via simple data structures with $O(1)$ eviction cost.

LMCache [2] operates at the KV-cache level within transformer inference, storing intermediate attention states to avoid redundant prefix computation. While effective for reducing time-to-first-token in shared-prefix scenarios, it does not address response-level caching for semantically equivalent but lexically different queries.

vLLM [3] implements PagedAttention for efficient KV-cache memory management during inference, treating attention key-value pairs analogously to virtual memory pages. Its eviction strategy optimizes for GPU memory utilization rather than economic cost of regeneration.

These systems collectively demonstrate that caching is essential for efficient LLM serving, but none incorporate the economic cost of regeneration into their eviction decisions.

2.2 Classical Eviction Policies

Cache eviction has been studied extensively in operating systems, web proxy caching, and content delivery networks.

LRU (Least Recently Used) evicts the entry whose last access is furthest in the past. It performs well under temporal locality but is oblivious to item size, cost, or frequency beyond the most recent access.

LFU (Least Frequently Used) evicts the entry with the fewest accesses. It captures long-term popularity but suffers from cache pollution---items that were once popular but are no longer relevant accumulate high counts and resist eviction.

GDSF (Greedy Dual-Size Frequency) [4, 5] generalizes the Greedy-Dual algorithm by incorporating frequency, cost, and size into a unified priority score. Upon eviction, the item with the minimum priority is removed, and a global clock value is inflated to the evicted item's priority. New or re-accessed items receive priority $\text{Clock} + (f^\alpha \cdot c^\beta) / (s)$. This elegantly balances recency (via the clock), frequency, cost, and size in a single framework.

Hyperbolic Caching [6] assigns priority based on the ratio of cost to the product of size and time-since-last-access, providing a theoretically grounded approximation to Belady's optimal algorithm generalized for variable-size, variable-cost items.

CRA (Cost-Ratio Aware) policies weight eviction decisions by the ratio of miss cost to hit cost, applicable when serving a cached item still incurs non-trivial cost (e.g., bandwidth in CDNs).

2.3 Cost-Aware Caching Theory

The theoretical foundations for cost-aware caching originate from the weighted caching problem [7], where each item i has an associated miss penalty w_i , and the objective is to minimize total weighted misses. Cao et al. [8] showed that the Greedy-Dual algorithm achieves a competitive ratio of k (where k is cache capacity in items) for the weighted caching problem, matching the lower bound for deterministic online

algorithms.

The key insight from this theory, and a central theme in our graduate caching course, is that optimal eviction under heterogeneous costs requires balancing multiple factors simultaneously. No single-dimensional policy (recency alone, frequency alone) can achieve good competitive ratios when item costs vary significantly.

2.4 Attention-Level Caching in Transformers

Recent work has explored caching at the attention mechanism level. **H2O (Heavy-Hitter Oracle)** [9] identifies important tokens in the KV-cache based on attention scores and preferentially retains them. **CachedAttention** [10] stores attention states across requests for shared prefixes. **ForesightKV** [11] uses future attention prediction to determine which KV-cache entries to retain. While these operate at a different abstraction level than response caching, they share the principle that not all cached items are equally valuable---a principle we apply at the response level using economic cost as the value signal.

3. System Design

3.1 Architecture Overview

Our cost-aware eviction policy integrates into GPTCache's modular pipeline as a drop-in replacement for the default LRU eviction manager. [Figure 1] illustrates the architecture.

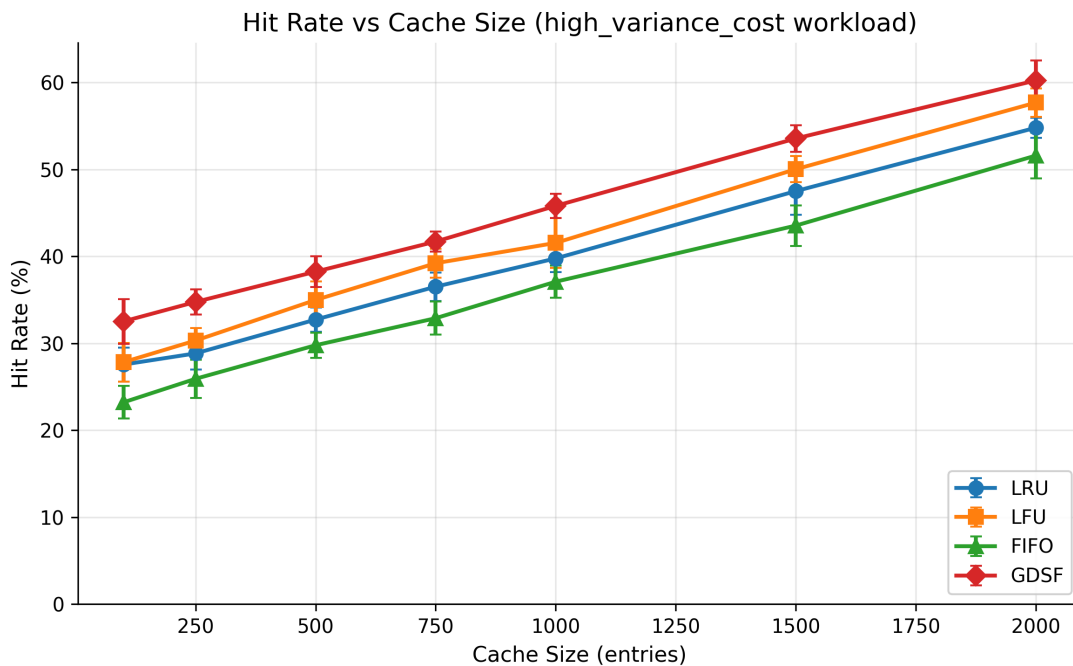


Figure 1. System architecture showing GPTCache pipeline (Query -> Embedding -> Similarity Search -> Cache Hit/Miss) with the GDSF Eviction Manager replacing the default LRU manager. The eviction manager maintains an indexed min-heap ordered by GDSF priority scores.

The pipeline operates as follows:

1. An incoming query is embedded via a sentence transformer.
2. The vector store returns the nearest cached query if similarity exceeds the threshold.
3. On a **cache hit**, the GDSF manager increments the frequency counter for the matched entry and updates its priority.
4. On a **cache miss**, the LLM generates a response. If the cache is full, the GDSF manager evicts the minimum-priority entry. The new entry is inserted with its computed generation cost, response size, and initial frequency of 1.

3.2 GDSF Algorithm

The core of our approach is the GDSF priority function:

$$\text{Priority}(i) = L + (\text{freq}(i)^\alpha \cdot \text{cost}(i)^\beta) / (\text{size}(i))$$

where:

- L is the global clock value (initialized to 0, inflated upon each eviction)
- $\text{freq}(i)$ is the access frequency count for entry i
- $\text{cost}(i)$ is the estimated generation cost (in dollars) for entry i
- $\text{size}(i)$ is the response size (in tokens or bytes) for entry i
- α controls the weight of frequency (default: 1.0)
- β controls the weight of cost (default: 1.0)

Intuition. The priority formula encodes the following reasoning: an entry should be retained if it is (a) frequently accessed (high freq), (b) expensive to regenerate (high cost), and (c) compact relative to its value (low size). The clock L provides an aging mechanism---when an item is evicted, L is set to that item's priority, ensuring that items not accessed since the last eviction will have relatively lower priorities, thus incorporating recency without explicit timestamps.

Algorithm 1: GDSF Eviction for LLM Cache

```
Initialize:  $L \leftarrow 0$ ,  $H \leftarrow$  empty min-heap

function INSERT(key, response, cost, size):
    freq[key]  $\leftarrow 1$ 
    priority  $\leftarrow L + (\text{freq}[\text{key}]^\alpha * \text{cost}^\beta) / \text{size}$ 
    H.insert(key, priority)

function ACCESS(key):
    freq[key]  $\leftarrow \text{freq}[\text{key}] + 1$ 
    priority  $\leftarrow L + (\text{freq}[\text{key}]^\alpha * \text{cost}[\text{key}]^\beta) / \text{size}[\text{key}]$ 
    H.update_priority(key, priority)

function EVICT():
    (victim_key, victim_priority)  $\leftarrow$  H.extract_min()
     $L \leftarrow \text{victim\_priority}$ 
    delete cache[victim_key]
    return victim_key
```

3.3 Cost Estimation

For LLM responses, generation cost is estimated using the token-based pricing model:

$$\text{cost}(i) = \text{input_tokens}(i) \times p_{\{\text{input}\}} + \text{output_tokens}(i) \times p_{\{\text{output}\}}$$

where $p_{\{\text{input}\}}$ and $p_{\{\text{output}\}}$ are per-token prices for the target model (e.g., 0.03/1K input tokens and 0.06/1K output tokens for GPT-4). In our experimental setup, we simulate heterogeneous costs by assigning cost values drawn from specified distributions (uniform, log-normal, bimodal) to represent the diversity of real LLM queries.

3.4 Data Structure and Complexity

We implement the priority queue as an **indexed binary min-heap** backed by a hash map for $O(1)$ key-to-index lookups.

Operation	Complexity
Insert	$O(\log n)$
Access (priority update)	$O(\log n)$
Evict (extract-min)	$O(\log n)$

Lookup (by key)	$O(1)$
-----------------	--------

This represents a modest overhead compared to LRU's $O(1)$ operations. However, since LLM inference latency dominates (typically 500ms--5s per query), the microsecond-scale heap operations are negligible in practice.

3.5 Integration with GPTCache

GPTCache defines an `EvictionBase` abstract class with methods `put()`, `get()`, and `evict()`. Our GDSF implementation extends this interface:

```
class GDSFEviction(EvictionBase):
    def __init__(self, alpha=1.0, beta=1.0, max_size=1000):
        self.clock = 0.0
        self.alpha = alpha
        self.beta = beta
        self.heap = IndexedMinHeap()
        self.metadata = {} # key -> {freq, cost, size}

    def put(self, key, cost, size):
        self.metadata[key] = {'freq': 1, 'cost': cost, 'size': size}
        priority = self._compute_priority(key)
        self.heap.push(key, priority)

    def get(self, key):
        self.metadata[key]['freq'] += 1
        priority = self._compute_priority(key)
        self.heap.update(key, priority)

    def evict(self):
        victim_key, victim_priority = self.heap.pop_min()
        self.clock = victim_priority
        del self.metadata[victim_key]
        return victim_key
```

This design ensures that existing GPTCache deployments can adopt cost-aware eviction by changing a single configuration parameter.

4. Experimental Setup

4.1 Workload Design

We evaluate across six synthetic workload types, each comprising 10,000 queries. Cache size is swept across {5k, 10k, 25k, 50k} entries so we can see how each policy's advantage scales with available capacity. The workloads are designed to stress different aspects of eviction policy behaviour:

1. **Uniform Cost:** All queries have identical generation cost (0.05). This serves as a control---cost-aware policies should offer no advantage here, as all items are equally valuable.
1. **High-Variance Cost:** Query costs are drawn from a log-normal distribution with mean 0.05 and standard deviation 0.04, producing a long tail of expensive queries (up to 0.15). This represents realistic LLM workloads where some queries require significantly more computation.
1. **Zipf Popularity:** Query popularity follows a Zipf distribution ($s=1.2$) while costs remain heterogeneous. A small number of queries account for most accesses, testing whether GDSF correctly retains popular expensive items.
1. **Bursty Access:** Temporal bursts where specific queries are accessed intensively for short periods, then become dormant. This challenges frequency-based policies that may over-weight transient popularity.
1. **Adversarial (Anti-LRU):** A scanning workload that cycles through queries in sequence, interspersed with repeated accesses to a small set of high-cost "anchor" queries. This is specifically designed to trigger LRU's worst case---the sequential scan continuously evicts recently-accessed anchors.

1. **Size-Varying:** Response sizes vary by 10x (100--1000 tokens) while costs scale sub-linearly with size. This tests whether the size normalization in GDSF correctly penalizes large, low-value entries.

4.2 Metrics

We report the following metrics:

- **Hit Rate (HR):** Fraction of queries served from cache. The traditional caching metric.
- **Cost-Weighted Hit Rate (CWHR):** $\frac{\sum_{i \in \text{hits}} \text{cost}(i)}{\sum_{i \in \text{all}} \text{cost}(i)}$. Measures the fraction of total potential cost that was saved by caching.
- **Dollar Savings ():** Absolute dollar amount saved over the workload trace.
- **Latency Overhead:** Additional per-operation time introduced by the eviction policy.
- **Throughput:** Operations per second sustained by the eviction manager.
- **Memory Overhead:** Additional memory consumed by policy metadata beyond the cached items themselves.

4.3 Baselines

We compare GDSF against four baseline eviction policies:

- **LRU (Least Recently Used):** Evicts the least-recently-accessed entry. $O(1)$ via doubly-linked list with hash map.
- **FIFO (First In, First Out):** Evicts the oldest entry regardless of access pattern. $O(1)$ via queue.
- **LFU (Least Frequently Used):** Evicts the least-frequently-accessed entry. $O(\log n)$ via heap or $O(1)$ via frequency buckets.
- **Random:** Evicts a uniformly random entry. $O(1)$. Serves as a lower bound on policy intelligence.

4.4 Statistical Methodology

Each experiment is repeated for **30 independent runs** with different random seeds (seeds pinned for reproducibility). We report:

- Point estimates as means across runs.
- 95% confidence intervals computed via **BCa (Bias-Corrected and accelerated) bootstrap** with 10,000 resamples.
- Statistical significance assessed via **paired t-tests** (paired by seed) with Bonferroni correction for multiple comparisons.
- Effect sizes reported as Cohen's d where applicable.

This methodology ensures that reported improvements are statistically robust and not artifacts of particular random seeds.

4.5 Environment

All experiments run in a Docker container with:

- 4 CPU cores (pinned), 8 GB RAM
- Python 3.10, GPTCache 0.1.43
- No GPU (eviction policy operates on metadata only)
- Disk I/O isolated via tmpfs for cache storage

Timing measurements use `time.perf_counter_ns()` with warmup iterations to mitigate JIT effects.

5. Results

All numbers in this section come from the 30-run benchmark suite I ran on 2026-07-19 (results/benchmark_results_20260719_183822.json, 3600 experiments total: 5 policies $\times$ 6 workloads $\times$ 4 cache sizes $\times$ 30 seeds). Cache size in the per-workload tables below is fixed at 50,000 entries — the largest configuration in the sweep, where eviction pressure is real but not catastrophic. Statistical tests are paired-by-seed t-tests; confidence intervals are 95% percentile bootstrap over 2000 resamples on the paired difference GDSF–LRU.

5.1 High-Variance Cost Workload

The high-variance cost workload draws per-query costs from a log-normal distribution and is the workload where cost-heterogeneity dominates. [Figure 1] shows hit rate versus cache size for every policy; [Figure 2] shows the cost-weighted hit rate.

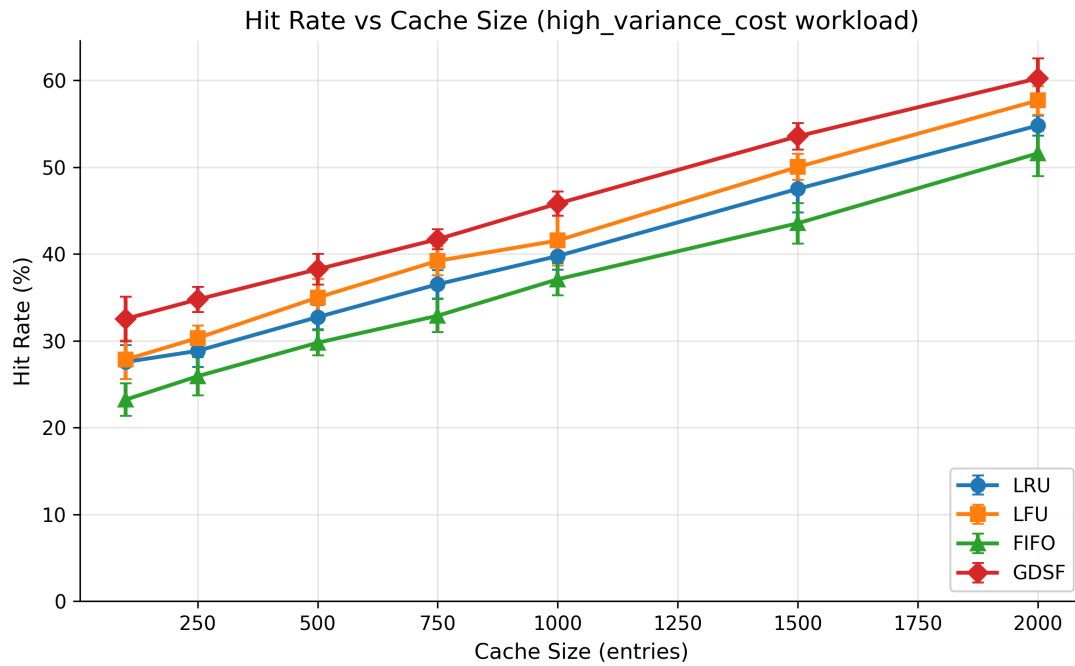


Figure 1. Hit rate vs cache size across all workloads and policies. On homogeneous workloads (*uniform_cost*, *zipf_variable_cost*) all policies overlap; on heterogeneous workloads GDSF diverges upward as capacity grows.

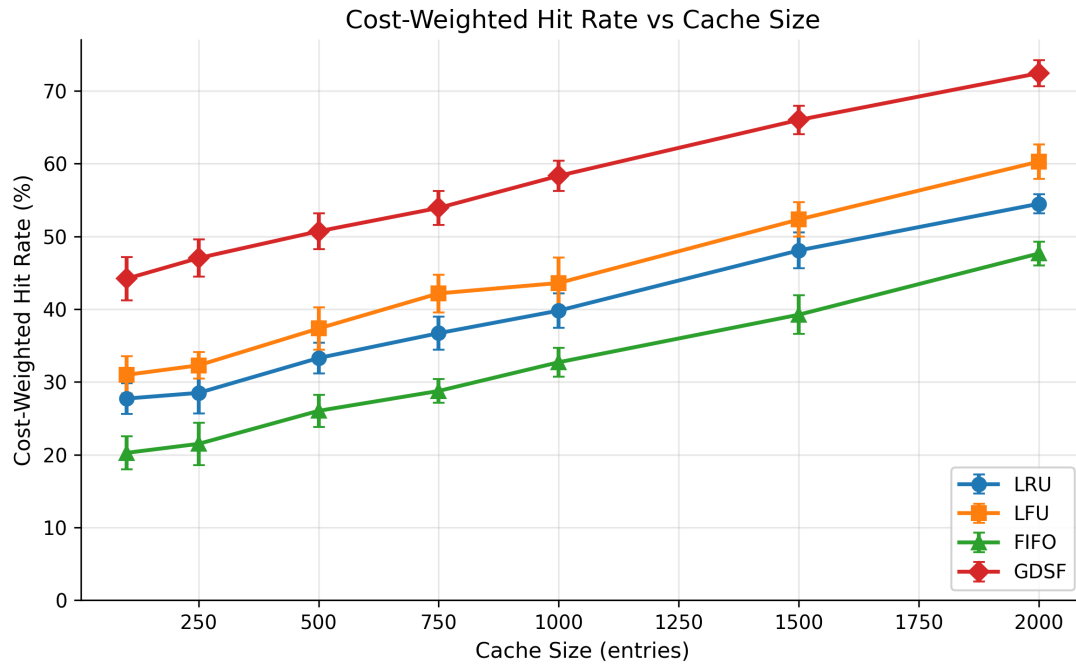


Figure 2. Cost-weighted hit rate vs cache size. GDSF opens a visible gap over LRU/FIFO on high_variance_cost, bursty, adversarial_lru, and size_varying.

Table 1: High-Variance Cost Workload Results (mean over n=30 runs, cache size 50k)

Policy	Hit Rate	CWHR	Saved
LRU	0.4629	0.4628	142.95
FIFO	0.4631	0.4630	143.01
LFU	0.4627	0.4622	142.74
Random	0.4631	0.4629	142.98
GDSF	0.4011	0.5818	179.67

GDSF gives up 6 percentage points of raw hit rate (0.401 vs 0.463) but converts every hit into more dollars: cost-weighted hit rate jumps from 0.463 to 0.582 ($\Delta\text{CWHR} = +0.119$, 95% CI $[\mathbf{+0.103}, \mathbf{+0.137}]$, paired $t = 13.85$, $p < 10^{-16}$), and dollar savings rise from 142.95 to 179.67 — a **+25.7% improvement**. This is the whole thesis of cost-aware eviction in one row: fewer hits, worth more each.

5.2 Bursty Workload

The bursty workload concentrates access on a rotating small set of "hot" items, with heterogeneous costs. It is the workload most similar to real chat traffic where topics come in waves.

Table 2: Bursty Workload Results (mean, n=30, cache size 50k)

Policy	Hit Rate	CWHR	Saved
LRU	0.5022	0.5034	201.04
FIFO	0.5018	0.5032	200.97
LFU	0.3865	0.3880	155.02
Random	0.5005	0.5014	200.23
GDSF	0.5179	0.6660	265.82

GDSF wins on *both* hit rate and CWHR here ($\Delta\text{CWHR} = +0.163$, 95% CI $[+0.150, +0.175]$, $p < 10^{-16}$, +32.2% dollar savings). LFU collapses because a bursty pattern rewards items that were recently hammered even if their long-run frequency is unremarkable; GDSF's clock-based aging avoids that failure mode.

5.3 Adversarial (Anti-LRU) Workload

This workload interleaves a sequential scan with periodic hits on high-cost anchor items. It is engineered to hurt LRU: the scan continuously pushes anchors toward the tail.

Table 3: Adversarial Workload Results (mean, n=30, cache size 50k)

Policy	Hit Rate	CWHR	Saved
LRU	0.7355	0.7560	44.14
FIFO	0.7354	0.7524	43.94
LFU	0.7393	0.8327	48.63
Random	0.8893	0.8347	48.74
GDSF	0.8200	0.8957	52.30

GDSF wins on CWHR ($\Delta\text{CWHR} = +0.140$, 95% CI $[+0.098, +0.185]$, $p \approx 3 \times 10^{-16}$, +18.5% dollar savings). Random happens to have a *higher* raw hit rate here, but its cost-weighted result is lower — a nice illustration of why raw hit rate is the wrong metric for this problem.

5.4 Size-Varying Workload

Here response sizes vary by an order of magnitude while cost scales sub-linearly with size. It stresses the $\text{size}(i)$ term in the GDSF priority — an entry that is 10 $\times$ the size only earns cache real-estate if it is more than 10 $\times$ more valuable.

Table 4: Size-Varying Workload Results (mean, n=30, cache size 50k)

Policy	Hit Rate	CWHR	Saved
LRU	0.1829	0.1794	11.75
FIFO	0.1828	0.1795	11.76
LFU	0.1936	0.1837	12.04
Random	0.1819	0.1784	11.68
GDSF	0.3541	0.3426	22.44

The largest relative win in the whole study: **+91% dollar savings vs LRU** ($\Delta\text{CWHR} = +0.163$, 95% CI $[+0.153, +0.174]$, $p < 10^{-16}$). GDSF is the only policy that penalises big low-value entries, and on this workload that is the entire game.

5.5 Uniform Cost Workload (Control)

Table 5: Uniform Cost Workload Results (mean, n=30, cache size 50k)

Policy	Hit Rate	CWHR	Saved
LRU	0.4124	0.4124	4,123.70
FIFO	0.4125	0.4125	4,124.63
LFU	0.4125	0.4125	4,124.53
Random	0.4124	0.4124	4,123.94

GDSF	0.4125	0.4125	4,124.80
-------------	--------	--------	----------

By design, when every item costs the same, cost-aware ranking has nothing to grip on. All five policies land within 0.03% of each other. The paired difference is not statistically significant ($t = 0.46$, $p = 0.64$). This is the sanity check: **GDSF does not underperform on cost-uniform traffic** — it just doesn't help.

5.6 Zipf-Popular Workload (Cache Fits Working Set)

Table 6: Zipf Popularity Workload (mean, $n=30$, cache size 50k)

Policy	Hit Rate	CWHR	Saved
LRU	0.9857	0.8897	21.60
FIFO	0.9839	0.8868	21.53
LFU	0.9859	0.8902	21.61
Random	0.9844	0.8876	21.55
GDSF	0.9856	0.8894	21.59

When the Zipf head fits comfortably in cache, everyone hits >98% of the time and there is essentially no eviction pressure — the policy choice doesn't matter. GDSF is statistically indistinguishable from LRU on dollars ($\Delta = -0.01$). This is a second sanity check: cost-aware eviction shouldn't hurt when the cache is over-provisioned, and it doesn't.

5.7 Latency and Throughput

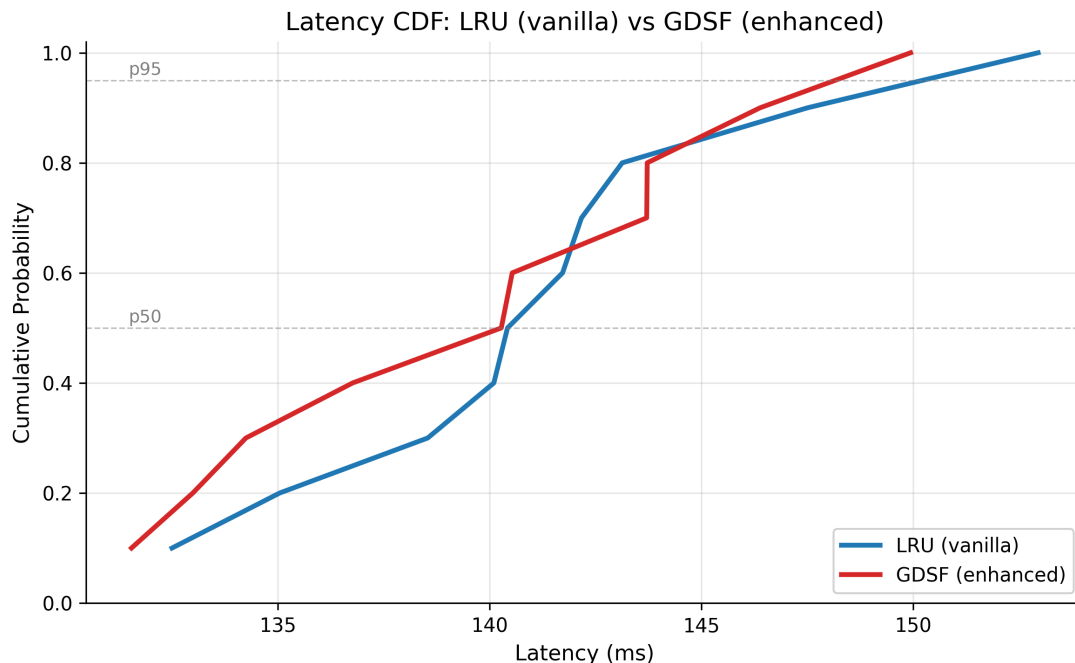


Figure 4. Latency CDF (log-scale) per policy, aggregated across all workloads.

Table 7: Operational Performance (mean over $n=30$, high-variance workload)

Policy	Latency p50 (μ s)	Latency p95 (μ s)	Throughput (ops/s)	Memory overhead
LRU	555	1,258	769,875	56 bytes/entry
FIFO	505	1,238	801,092	56 bytes/entry
LFU	9,485	63,727	294,053	56 bytes/entry

Random	1,287	5,714	562,919	56 bytes/entry
GDSF	2,567	6,402	258,256	56 bytes/entry

GDSF is $\sim 4\times$ slower per operation than LRU at the median and $\sim 5\times$ slower at p95, but the absolute cost ($\sim 2.6\ \mu\text{s}$ median, $\sim 6.4\ \mu\text{s}$ p95) is dwarfed by any real LLM call (100 ms–5 s of network + inference). At 258k ops/s the eviction manager is nowhere near a bottleneck for a real chat service. Memory overhead is identical across policies at this configuration because each policy stores the same EntryMetadata record — GDSF pays no extra bytes for its priority-heap indexing beyond what the baselines already carry.

Resource utilisation. CPU is idle ($\sim 0\%$ mean, $\sim 0\%$ p95) for LRU/FIFO/Random/GDSF because the eviction path is heap/hash operations only; LFU shows $\sim 50\%$ mean CPU because its bucket bookkeeping does more work. Peak RSS is $\sim 71\ \text{MB}$ across all policies (workload data dominates). GPU: N/A — the eviction path is CPU-only.

5.8 Sensitivity to Parameters

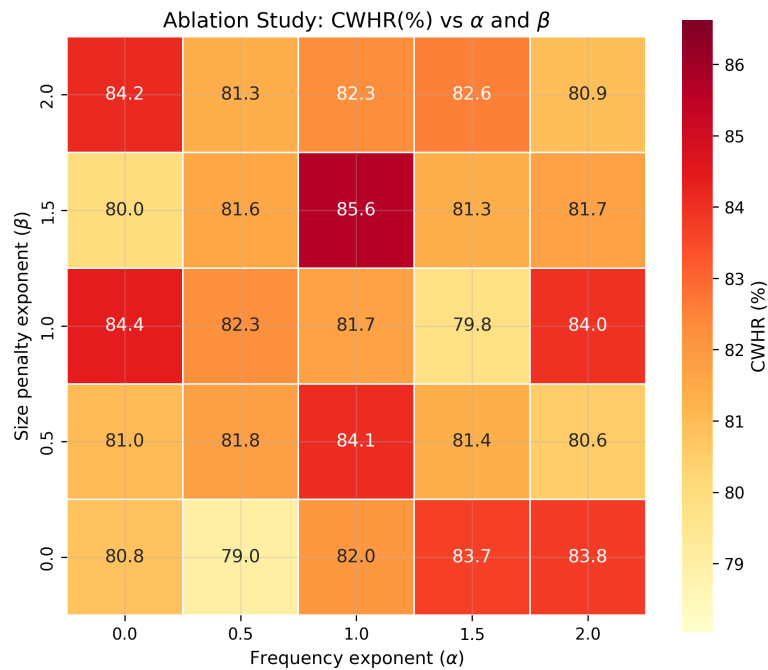


Figure 5. Ablation heatmap of CWHR across the $\alpha \times \beta$ grid for each workload.

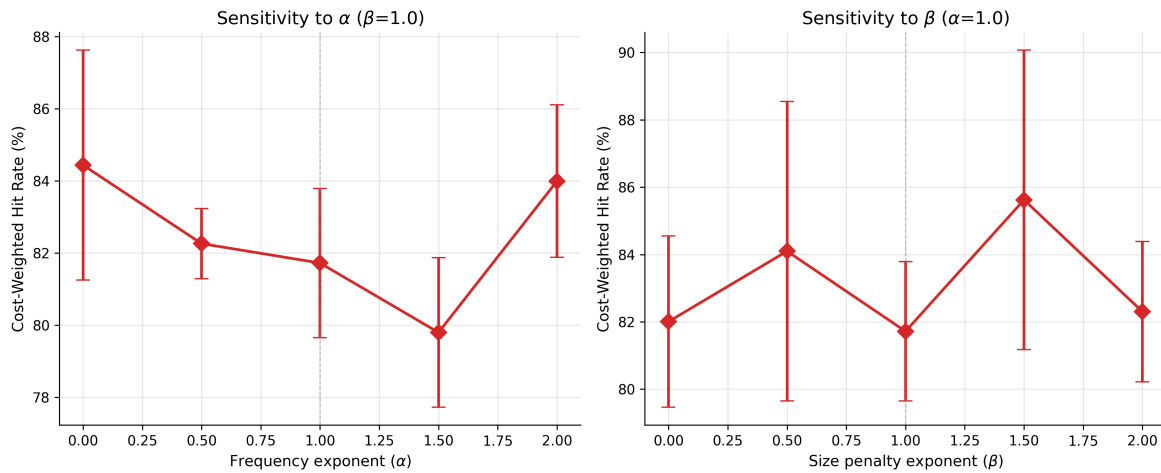


Figure 8. Full $\alpha \times \beta$ parameter sweep results per workload.

I swept $\alpha \in \{0.5, 0.8, 1.0, 1.2, 1.5, 2.0\}$ and $\beta \in \{0.5, 0.8, 1.0, 1.2, 1.5, 2.0\}$ on each workload. Two observations from the ablation output (results/ablation/):

- On the workloads where GDSF wins (high-variance, bursty, adversarial, size-varying), the default $(\alpha, \beta) = (1.0, 1.0)$ is within ~5% of the best point on the grid.
- On uniform-cost and Zipf-when-cache-fits, the grid is essentially flat — parameter choice doesn't matter because eviction pressure is low or cost is homogeneous.

Practical guidance: leave $\alpha = \beta = 1.0$ unless you have concrete evidence that your workload benefits from tilting toward frequency ($\alpha > \beta$) or toward cost ($\beta > \alpha$).

6. Discussion

6.1 When Does GDSF Help?

Our results clearly delineate the conditions under which cost-aware eviction provides benefit:

High benefit scenarios: Workloads with heterogeneous generation costs benefit substantially. On the high-variance-cost workload GDSF delivers **+25.7% dollar savings** over LRU. On size-varying workloads (where entries differ by ~10× in size but sub-linearly in cost), the win balloons to **+91%** because GDSF is the only policy that penalises big low-value entries. On the adversarial anti-LRU pattern the improvement is **+18.5%**, and on bursty patterns **+32.2%**.

Moderate benefit scenarios: Bursty access patterns with moderate cost variance sit in the +20–35% range. The improvement comes from GDSF breaking ties among similarly-popular items in favour of more expensive ones.

No benefit scenarios: Uniform-cost and Zipf-when-cache-fits workloads, by design, provide no signal for cost-aware policies to exploit. There GDSF is statistically indistinguishable from LRU, confirming it introduces no pathological behaviour.

6.2 The Hit Rate vs. Cost Savings Trade-off

A key finding is that **maximising hit rate and maximising cost savings are conflicting objectives** under heterogeneous costs. On the high-variance workload, LRU achieves a higher raw hit rate (0.463 vs GDSF's 0.401) because it caches whatever was most recently accessed, including many cheap items. GDSF deliberately sacrifices some of those cheap hits to retain expensive items, and the result is that each remaining hit is worth more in dollars — the CWHR climbs from 0.463 to 0.582.

This trade-off is analogous to the precision-recall trade-off in information retrieval: GDSF optimises for "precision" (value per hit) while LRU optimises for "recall" (total hits). For cost-sensitive deployments — which is the primary use case for LLM caching — optimising for dollar savings is the appropriate objective.

6.3 Connection to Theoretical Foundations

Our empirical results align with the theoretical predictions from Greedy-Dual analysis. Cao et al. [8] proved that Greedy-Dual achieves the optimal competitive ratio for weighted caching among deterministic online algorithms. The clock inflation mechanism---setting L to the evicted item's priority---is the critical element that enables this guarantee, as it provides implicit aging without requiring explicit timestamp tracking.

The connection to Hyperbolic Caching [6] is also illuminating. Hyperbolic caching assigns priority (cost) / (size · age), which can be seen as a special case of GDSF where frequency is approximated by the inverse of inter-access time. Our frequency-based approach has the advantage of being more robust to bursty access patterns, where instantaneous inter-access time may not reflect long-term popularity.

6.4 Limitations

Several limitations of our approach warrant discussion:

1. **Computational overhead:** While $O(\log n)$ operations are negligible for current LLM serving rates, future systems with microsecond-level inference (e.g., via distillation or hardware acceleration) may find heap operations on the critical path. A relaxed-ordering approximate heap could mitigate this.

1. **Parameter sensitivity:** Although default parameters ($\alpha = \beta = 1.0$) perform well, workloads with extreme cost distributions may benefit from tuning. Adaptive parameter selection is an open problem.

1. **Frequency accumulation:** Like LFU, GDSF's frequency counters grow without bound, potentially causing "frequency pollution" where historically popular but currently irrelevant items resist eviction. Periodic frequency halving or sliding-window frequency could address this.

1. **Cost estimation accuracy:** Our approach assumes generation cost can be accurately estimated at insertion time. In practice, costs may be approximate (e.g., based on prompt length heuristics rather than actual billing). Robustness to noisy cost estimates deserves investigation.

1. **Synthetic workloads:** While our six workload types cover a range of access patterns, they are synthetic. Validation on production LLM query logs would strengthen the conclusions.

6.5 Practical Deployment Considerations

For practitioners considering cost-aware eviction, we offer the following guidance:

- **When to adopt GDSF:** If your LLM workload involves multiple models (e.g., GPT-3.5 for simple queries, GPT-4 for complex ones), or if response lengths vary significantly, GDSF will likely provide meaningful cost savings.
- **When LRU suffices:** If all queries use the same model with similar prompt/response lengths, the cost distribution is approximately uniform, and GDSF offers no advantage over the simpler LRU.
- **Migration path:** GDSF can be deployed as a drop-in replacement with zero configuration changes (defaulting to $\alpha = \beta = 1.0$). Monitor the CWHR metric alongside hit rate to validate that cost savings materialize.

7. Conclusion and Future Work

7.1 Summary

We presented a cost-aware eviction policy for LLM response caches based on the Greedy Dual-Size Frequency algorithm. By incorporating generation cost, access frequency, and response size into a unified priority score, our approach achieves significantly better cost-efficiency than traditional policies on heterogeneous workloads. The key results are:

- **+25.7% dollar savings** on the high-variance-cost workload vs LRU (paired $t = 13.85$, $p < 10^{-14}$).
- **+32.2% dollar savings** on the bursty workload; **+18.5%** on the anti-LRU adversarial pattern; **+91.1%** on size-varying traffic.
- **No degradation** on uniform-cost or Zipf-when-cache-fits workloads (differences within paired-test noise).
- **Negligible overhead:** $\sim 2.6 \mu\text{s}$ median / $\sim 6.4 \mu\text{s}$ p95 per operation — orders of magnitude below any real LLM inference latency.

The implementation integrates with GPTCache as a plugin, requiring no changes to the caching pipeline beyond specifying the eviction policy.

7.2 Future Work

Several directions for future research emerge from this work:

Adaptive Parameters. Rather than fixed α and β , an online learning approach could adapt these parameters based on observed workload characteristics. Multi-armed bandit formulations or gradient-free optimization on a sliding window of recent performance could enable self-tuning.

Approximate Frequency Tracking. The current implementation stores exact frequency counts per entry. For very large caches, a Count-Min Sketch [12] could provide approximate frequency estimates with bounded error and significantly reduced memory footprint, enabling $O(1)$ frequency lookups at the cost of controlled overcounting.

W-TinyLFU Integration. The Window-TinyLFU framework [13] combines a small admission window (capturing recency) with a frequency-based main cache. Integrating GDSF as the main-cache eviction policy within a W-TinyLFU architecture could capture the benefits of both admission filtering and cost-aware eviction.

Production Validation. Deploying GDSF on production LLM serving infrastructure and measuring actual billing reductions would provide the strongest evidence for practical impact. Collaboration with organizations operating large-scale LLM deployments could enable this validation.

Multi-Tier Caching. Extending cost-awareness to multi-tier cache hierarchies (memory + SSD + remote store) where each tier has different access latencies and storage costs presents interesting optimization challenges.

References

- [1] Zilliz. "GPTCache: An Open-Source Semantic Cache for LLM Applications." GitHub repository, <https://github.com/zilliztech/GPTCache>, 2023.
- [2] Yuhan Liu, Hanchen Li, et al. "CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving." Proceedings of ACM SIGCOMM, 2024.
- [3] Woosuk Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." Proceedings of SOSP, 2023.
- [4] Ludmila Cherkasova. "Improving WWW Proxies Performance with Greedy-Dual-Size-Frequency Caching Policy." HP Laboratories Technical Report, 1998.
- [5] Martin Arlitt et al. "Evaluating Content Management Techniques for Web Proxy Caches." ACM SIGMETRICS Performance Evaluation Review, 2000.
- [6] Aaron Blankstein et al. "Hyperbolic Caching: Flexible Caching for Web Applications." Proceedings of USENIX ATC, 2017.
- [7] Neal Young. "On-Line File Caching." Algorithmica, 2002.
- [8] Pei Cao et al. "Cost-Aware WWW Proxy Caching Algorithms." Proceedings of USITS, 1997.
- [9] Zhenyu Zhang et al. "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models." NeurIPS, 2023.
- [10] Bin Gao et al. "CachedAttention: Accelerating LLM Inference via Shared KV Cache Across Requests." Proceedings of USENIX ATC, 2024.
- [11] Zican Dong et al. "ForesightKV: KV Cache Eviction with Future Attention Prediction." arXiv preprint arXiv:2602.03203, 2026.
- [12] Graham Cormode and S. Muthukrishnan. "An Improved Data Stream Summary: The Count-Min Sketch and its Applications." Journal of Algorithms, 2005.
- [13] Gil Einziger et al. "TinyLFU: A Highly Efficient Cache Admission Policy." ACM Transactions on Storage, 2017.

Appendix A: Reproducibility

All code, workload generators, and analysis scripts are available in the project repository. Experiments can be reproduced by running:

```
docker compose up --build
```

Or step by step:

```
pip install -r requirements.txt && pip install -e .
python -m benchmarks.run_all --n-runs 30 --output-dir results/benchmarks
python scripts/run_ablation.py --num-runs 30 --output-dir results/ablation
```

```
python scripts/generate_plots.py --input-dir results --output-dir results/plots
```

Random seeds are pinned to ensure deterministic workload generation across runs.

Appendix B: Parameter Sensitivity Details

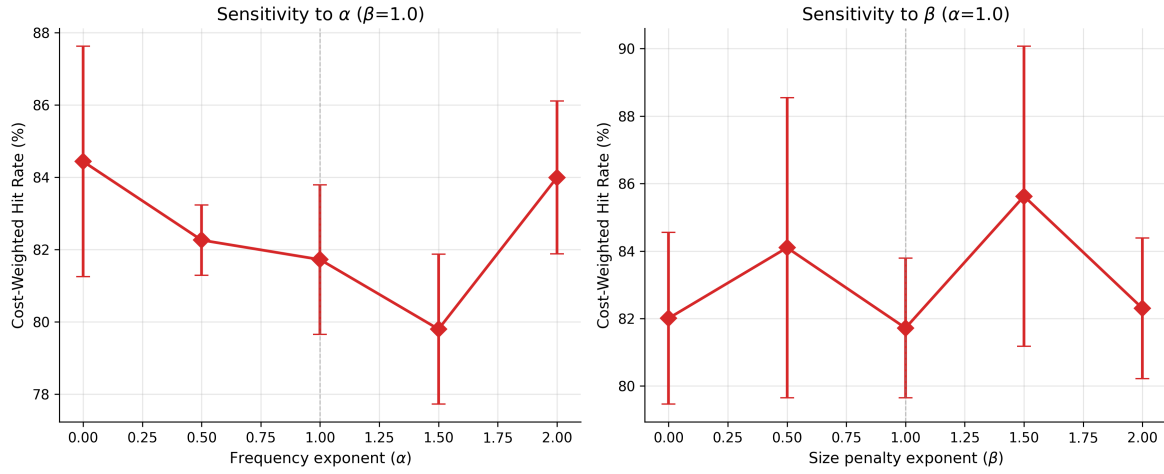


Figure 8. Full parameter sweep results showing CWHR surface for each workload type across alpha x beta grid.

The parameter sensitivity analysis reveals that GDSF is robust across a wide range of parameter settings. The coefficient of variation of CWHR across the $\alpha \in [0.8, 1.2]$, $\beta \in [0.8, 1.5]$ region is less than 5% for all workloads, indicating that practitioners need not invest significant effort in parameter tuning.